

Design Design Using Verilog and FPGA flow using Vivado

Assignment 1

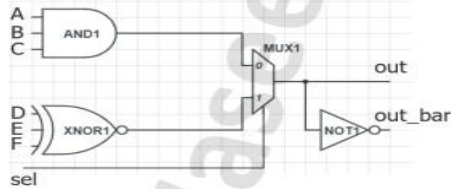
Combinational Circuit Design

NAME : Salah Adel Mohamed Elsadek

Assignment

1)

- The design has 7 inputs and 2 outputs
- Use assign statements to design the following

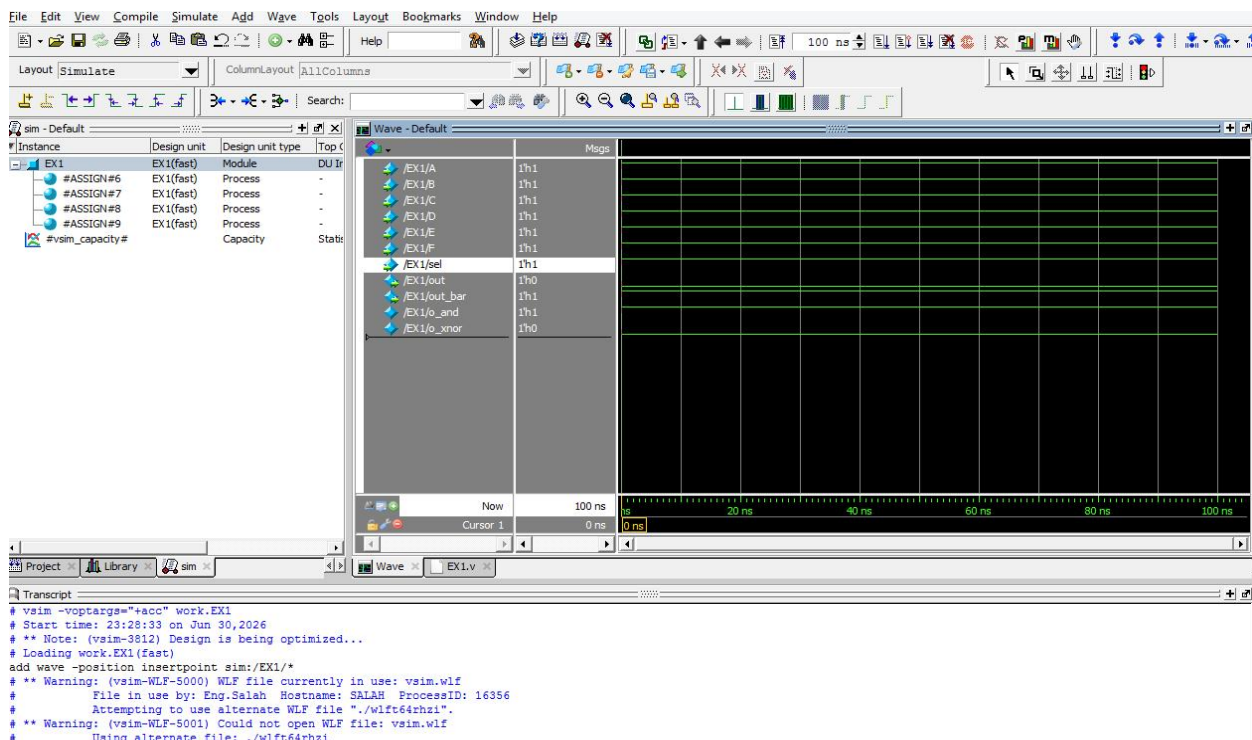


Verilog code

```
module EX1 (
    input A, B, C, D, E, F, sel,
    output out, out_bar
);
    wire o_and, o_xnor;
    assign o_and = A & B & C;
    assign o_xnor = ~(D ^ E ^ F);
    assign out = (sel==1)? o_xnor: o_and ;
    assign out_bar = ~out;
endmodule
```

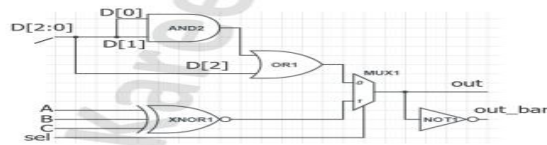
Waveform

At (A,B,C,D,E,F=1 AND sel=1) expected out= 0 , out_bar =1



2)

- The design has 5 inputs and 2 outputs
 - Inputs
 - D -> width = 3
 - A, B, C, Sel -> width = 1
 - Outputs
 - Out, out_bar -> width = 1
- Use Behavioral coding style to implement the following



Verilog code

```
module EX2 (
input [2:0] D,
input A,B,C, sel,
output reg out,
output reg out_bar
);
reg o_and , o_or , o_xnor;
always @(*) begin
o_and = D[0] & D[1];
o_or = o_and | D[2];
o_xnor = ~(A ^ B ^ C);

out = (sel) ? o_xnor : o_or;

out_bar = ~out;end

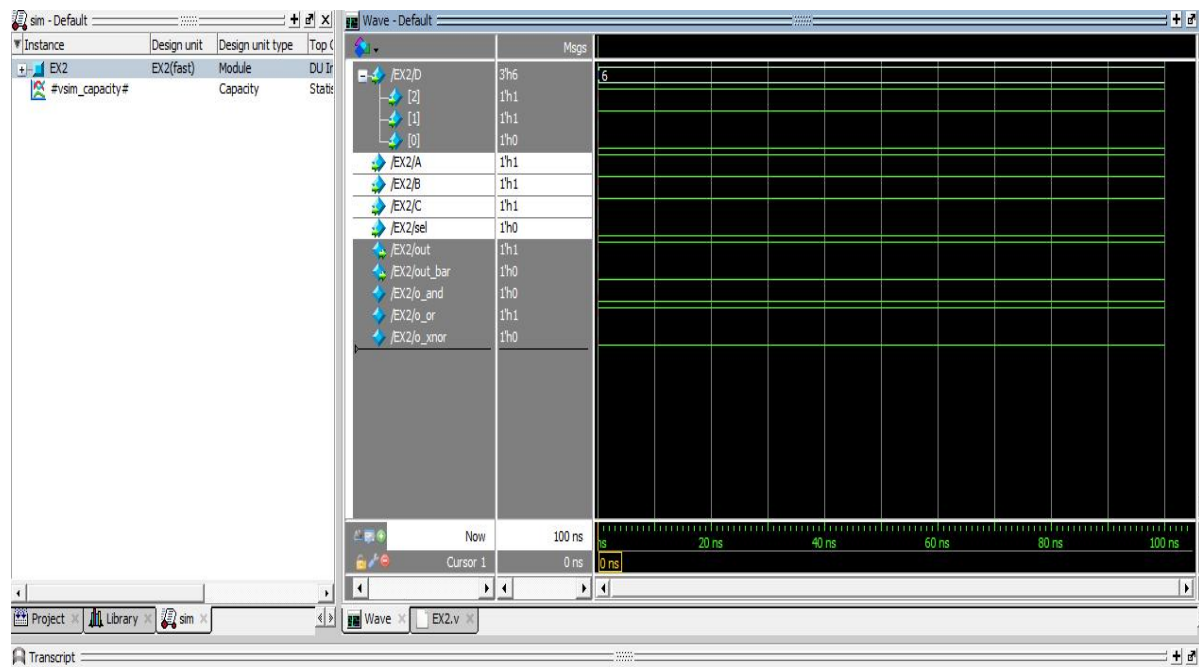
endmodule
```

Waveform

At (D0=0,D1=1,D2=1 => o_or =1)

(A,B,C = 1 => o_xnor=0)

@Sel=0 => expected out= 1 , out_bar =0



3) Implement 4-bit adder using addition operator and assign statement

- The design takes 2 inputs (A, B) and the summation is assigned to output (C) ignoring the carry

Verilog code

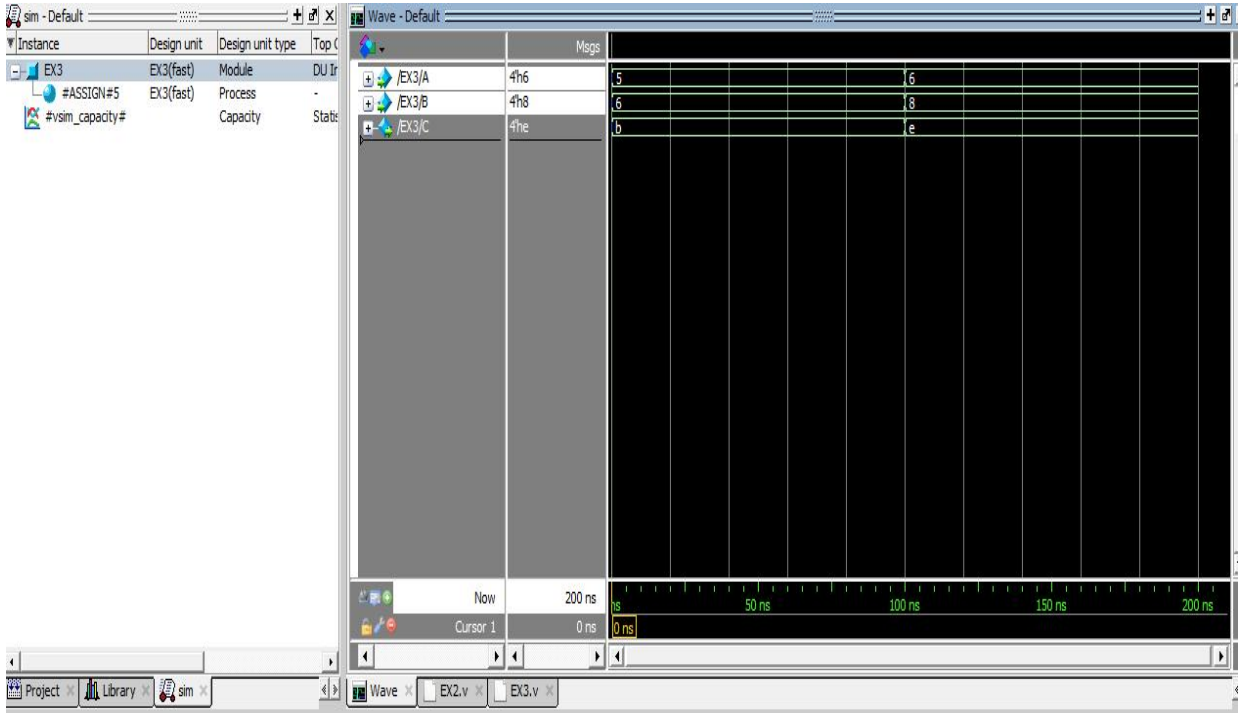
```

module EX3 (
  input  [3:0] A, B,
  output [3:0] C
);
  assign C = A + B ;

endmodule

```

Waveform



4) Implement 2-to-4 Decoder using conditional operator (A logic decoder has n input lines and 2^n output lines. Each output line corresponds to a unique combination of the input values.)

- The design has input **A** (2 bits) and output **D** (4 bits)
- you can use the following format for the conditional operator.
- assign <output_signal> = <condition1> ? <value1> : <condition2> ? <value2> : <default_value>;

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Verilog code

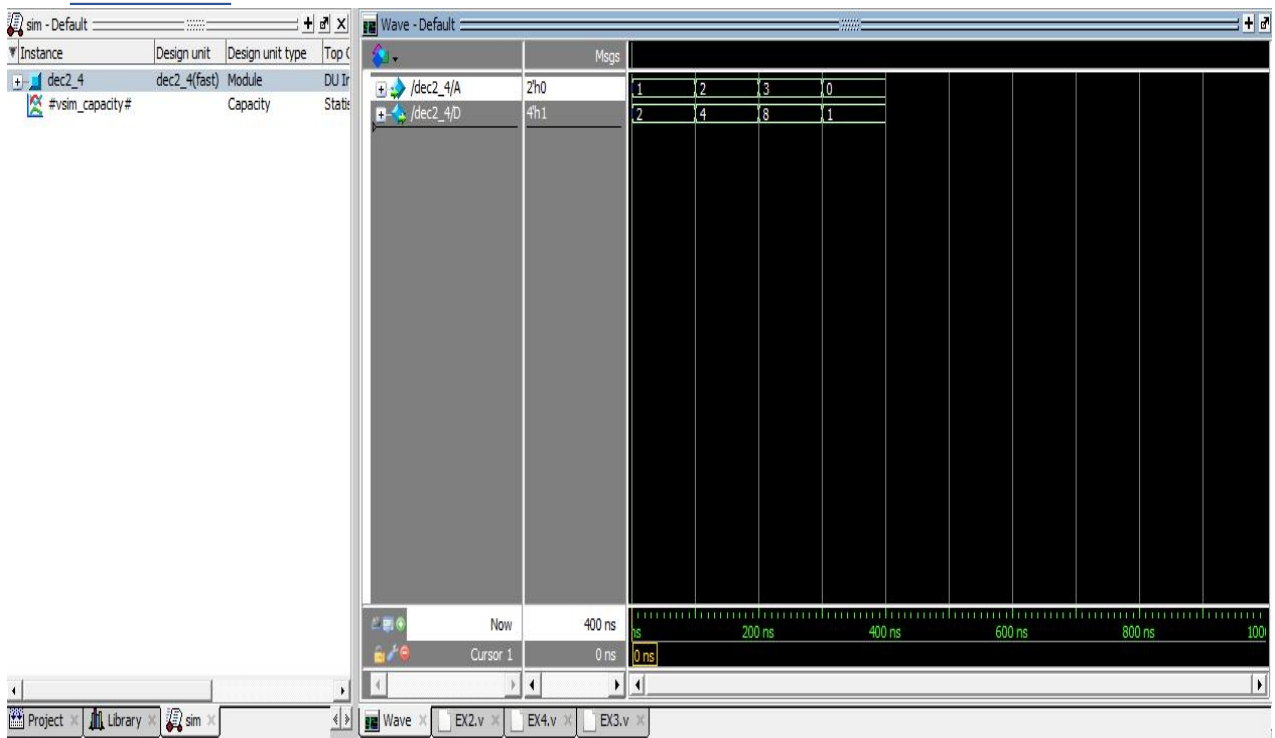
```

module dec2_4 #(parameter n=2) (
    input [n-1:0] A,
    output [2*n -1:0] D
);

assign D = (A==2'b00)? 4'b0001 : (A==2'b01)? 4'b0010 :
(A==2'b10)? 4'b0100 : 4'b1000 ;
endmodule

```

Waveform



5) Implement an even parity generator module using assign statement. In case you don't know what a parity bit is, please check this [link](#). The design input is a bus where a reduction operator will be used to generate the even parity bit.

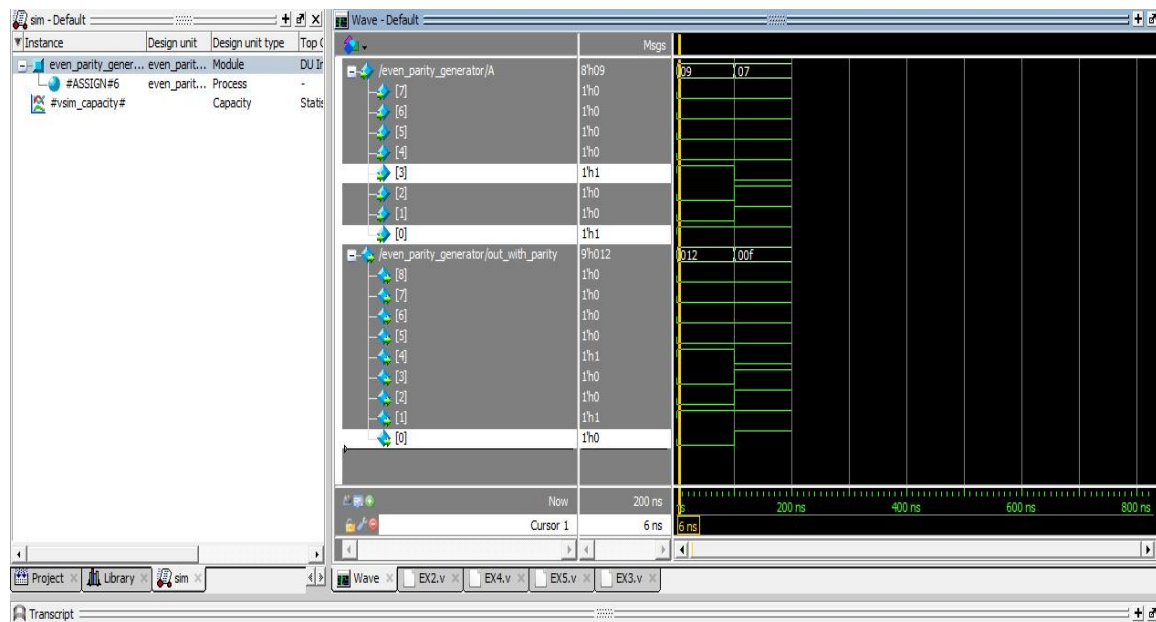
- The design has 1 input **A** (8 bits) and 1 output **out_with_parity** (9 bit) where the parity bit calculated will be inserted in the least significant bit of the output bus and the remaining bits will be the input A (Hint: use concatenation).

Verilog code

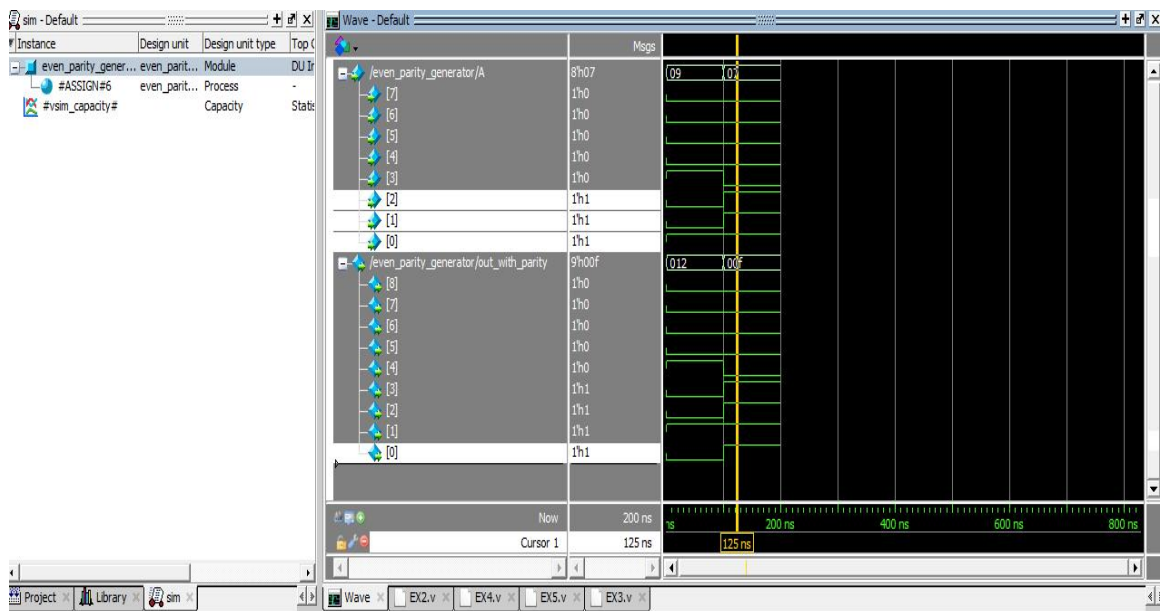
```
module even_parity_generator(  
    input  [7:0] A,  
    output [8:0] out_with_parity  
);  
  
    assign out_with_parity = {A, ^A};  
  
endmodule
```

Waveform

1- No. Of 1's even => parity bit = 0

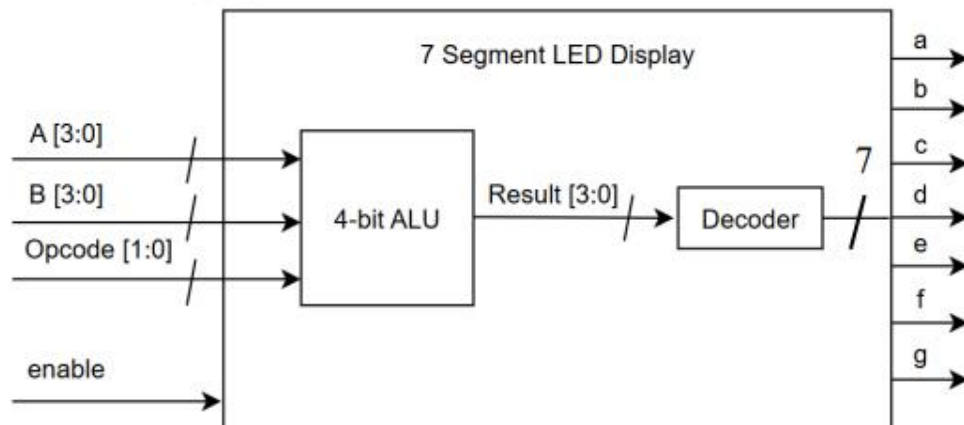


2-No. Of 1's odd => parity bit = 1



6)

In this design, you will **connect your 4-bit ALU (done in the lab) to a 7-segment display** so that the result of the ALU operation is shown as a **hex digit (0-F)**.



Verilog code

```
module S_segment_display #(parameter N=4 ) (
input [N-1 : 0] A , B ,
input [1 : 0] opcode ,
input enable ,
output reg a, b, c, d, e, f, g

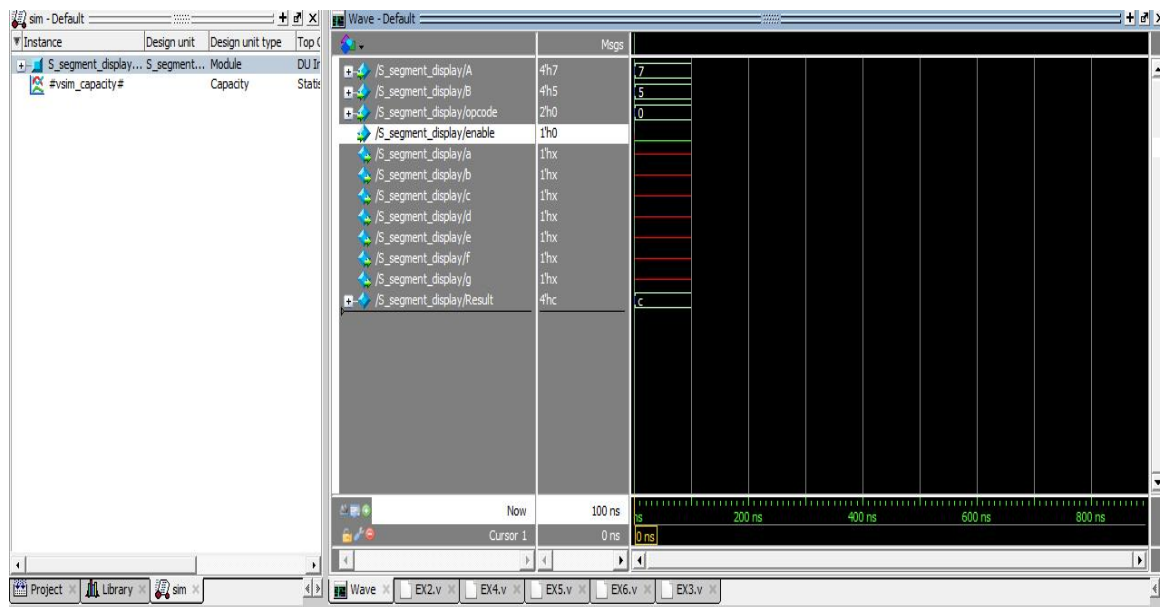
);
wire [N-1 : 0] Result;
ALU_N_BIT #(N(N)) ALU
( . in0(A), . in1(B), . opcode(opcode), . out(Result) );

always @(*)begin
if(!enable)
{a, b, c, d, e, f, g} = 7'b x ;
else begin
case(Result)
4'h0: {a, b, c, d, e, f, g} = 7'b1111110;
4'h1: {a, b, c, d, e, f, g} = 7'b0110000;
4'h2: {a, b, c, d, e, f, g} = 7'b1101101;
4'h3: {a, b, c, d, e, f, g} = 7'b1111001;
4'h4: {a, b, c, d, e, f, g} = 7'b0110011;
4'h5: {a, b, c, d, e, f, g} = 7'b1011011;
4'h6: {a, b, c, d, e, f, g} = 7'b1011111;
4'h7: {a, b, c, d, e, f, g} = 7'b1110000;
4'h8: {a, b, c, d, e, f, g} = 7'b1111111;
4'h9: {a, b, c, d, e, f, g} = 7'b1111011;
4'hA: {a, b, c, d, e, f, g} = 7'b1110111;
4'hB: {a, b, c, d, e, f, g} = 7'b0011111;
4'hC: {a, b, c, d, e, f, g} = 7'b1001110;
4'hD: {a, b, c, d, e, f, g} = 7'b0111101;
4'hE: {a, b, c, d, e, f, g} = 7'b1001111;
4'hF: {a, b, c, d, e, f, g} = 7'b1000111;
default: {a, b, c, d, e, f, g} = 7'b0;
endcase
end

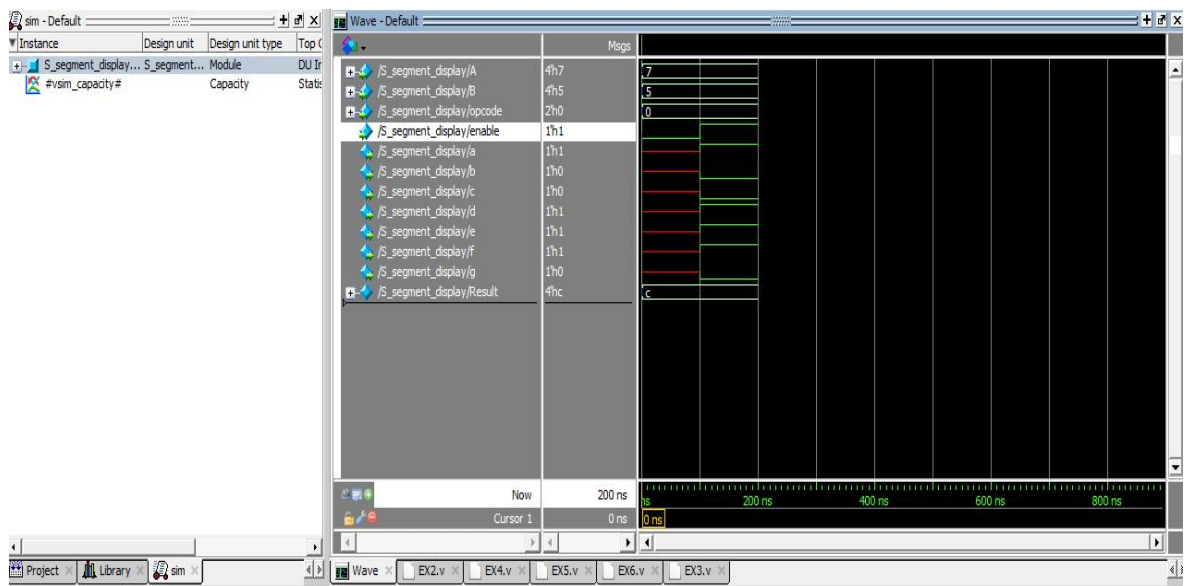
endendmodule
```

Waveform

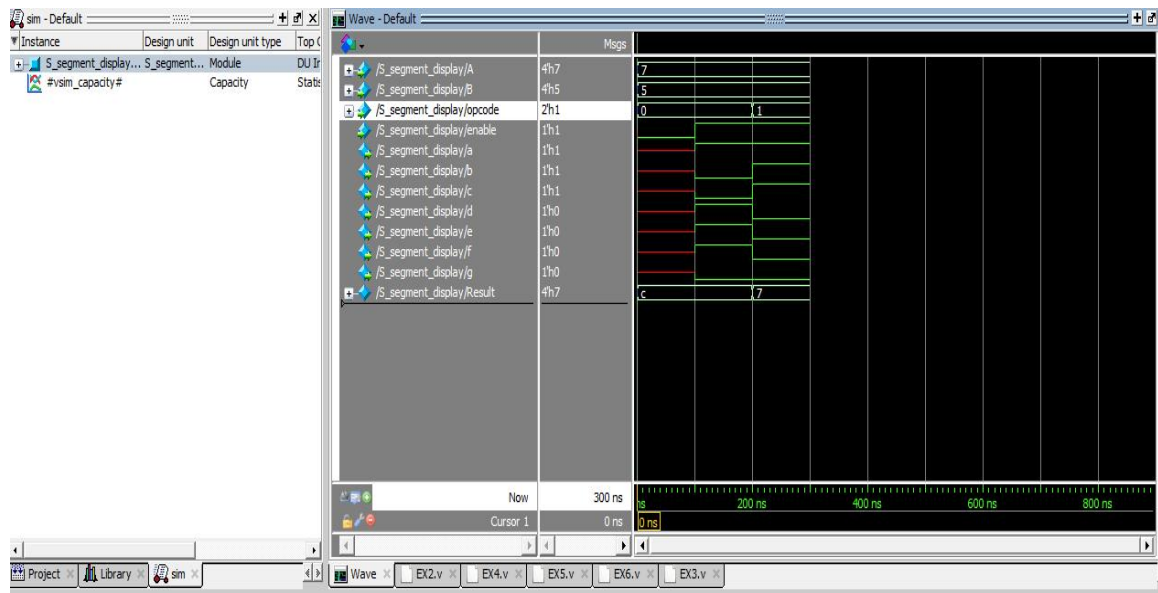
1- (enable = 0) => out = x



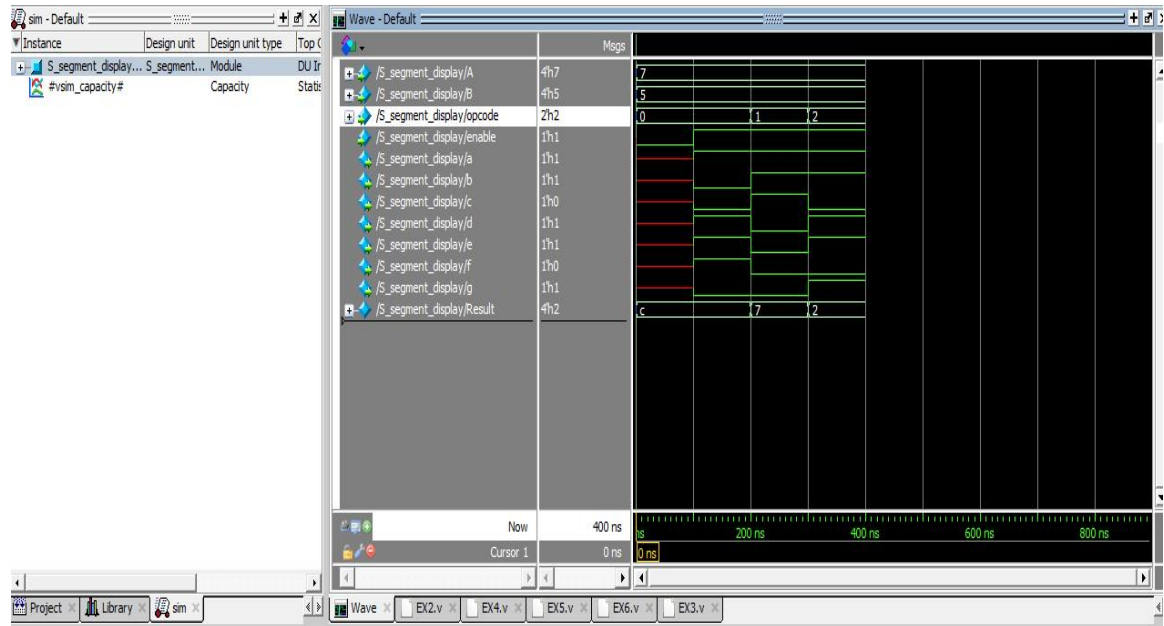
2- (enable = 1 , A= 7 , B = 5 , opcode=0) => addation opration so display C on 7 segment



3- (enable = 1 , A= 7 , B = 5 , opcode=1) => OR logic operation so display 7 on 7 segment



4- (enable = 1 , A= 7 , B = 5 , opcode=2) => subtraction operation so display 2 on 7 segment



5- (enable = 1 , A= 7 , B = 5 , opcode=3) => xor logic operation so display 2 on 7 segment

